

Agenda

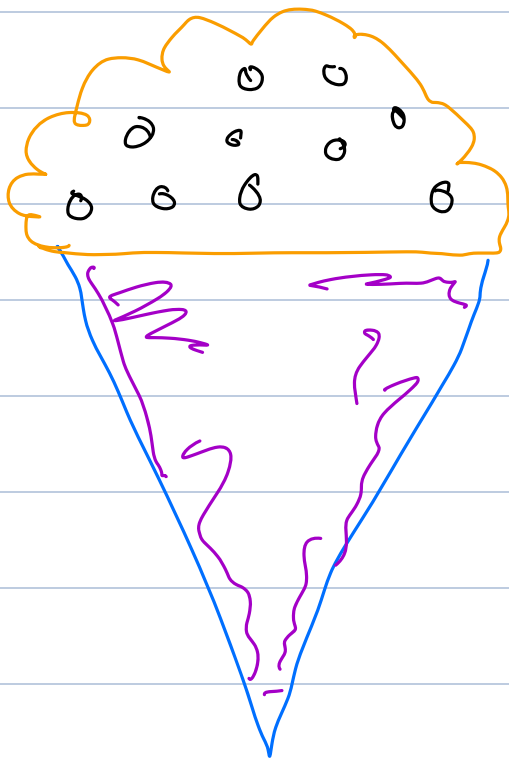
- Structural {
- Decorator
 - Flyweight

1] Decorator

Imagine you are working at an icecream company (Baskin's n Robbins).

You want to do 2 things:

- ① Get cost of the icecream
- ② Get description of the icecream.



cost of this
icecream =

cost of cone +
cost of mango scoop +
cost of choco chips +
cost of strawberry syrup

Approach #1:

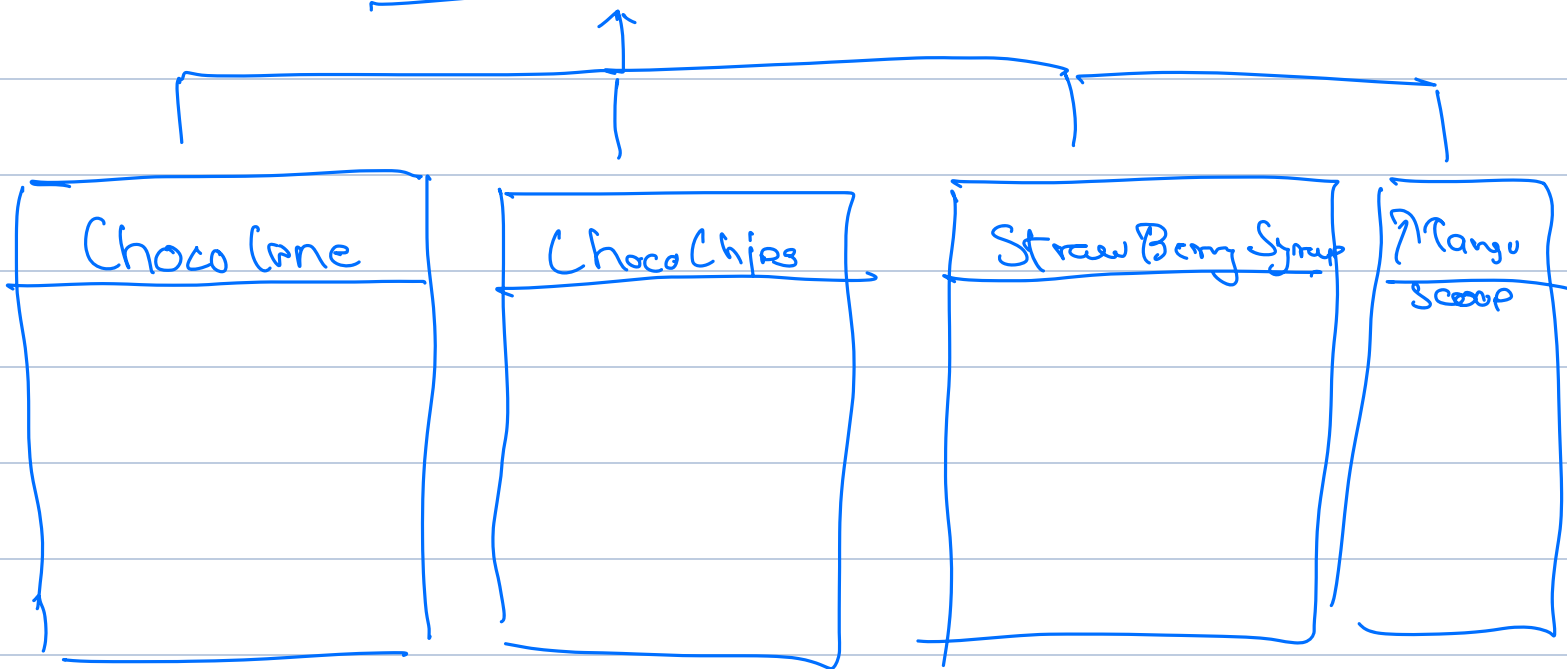
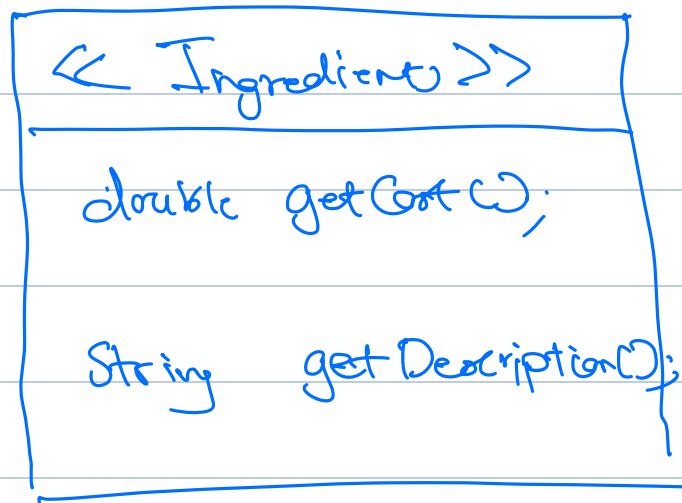
We generate diff classes for all possible permutations & combinations of icecreams.

Flavours of cones $\times$ (Flavours of scoops) $\times$ (Flavours of add-ons)

$\nearrow \quad \nwarrow$
 Syrups Drip

Case: ① Class explosion

Approach #2:



Ingredients

Cone

Scoop

Syrup

Chips

compulsory

Addons

```

class ChocoChips implements Ingredients {
    public ChocoChip (Ingr bI) {
        this.bI = bI
    }
}

```

Ingredients baseIngredient;

```

double getCost () {

```

```

    return baseIngredient.getCost()
        + 20;
}

```

```

String getDescription () {

```

3

3

```

new ChocoChips (

```

```

new Strawberry Syrup (

```

```

new Mango Scoop (

```

```

new Choco Cone ()

```

```

)

```

```

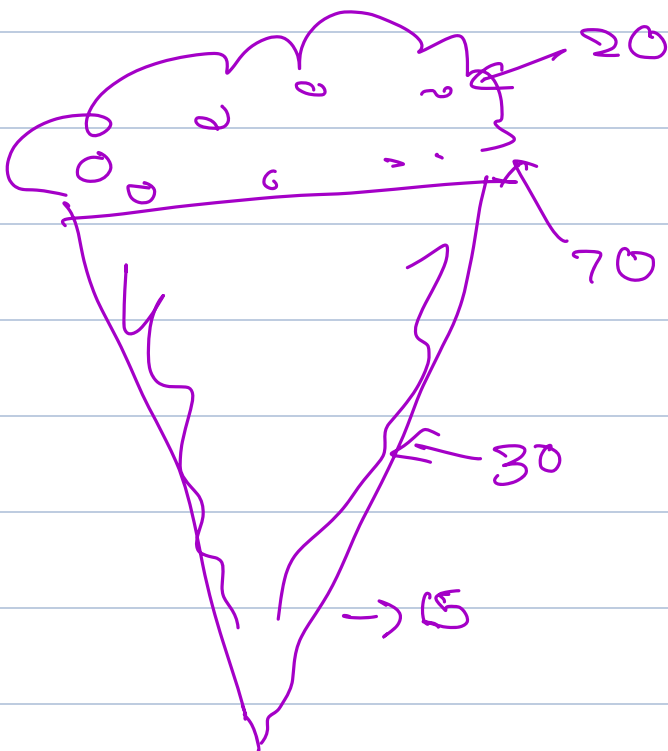
)

```

```

)

```



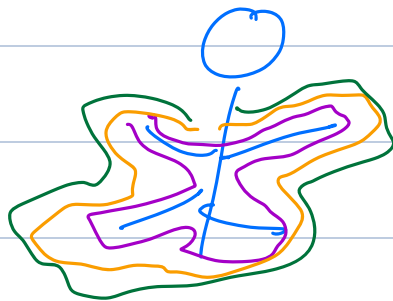
```
class ChocoConeimport Togr {
```

```
    double getCost () {
```

```
        return 15;
```

```
    }
```

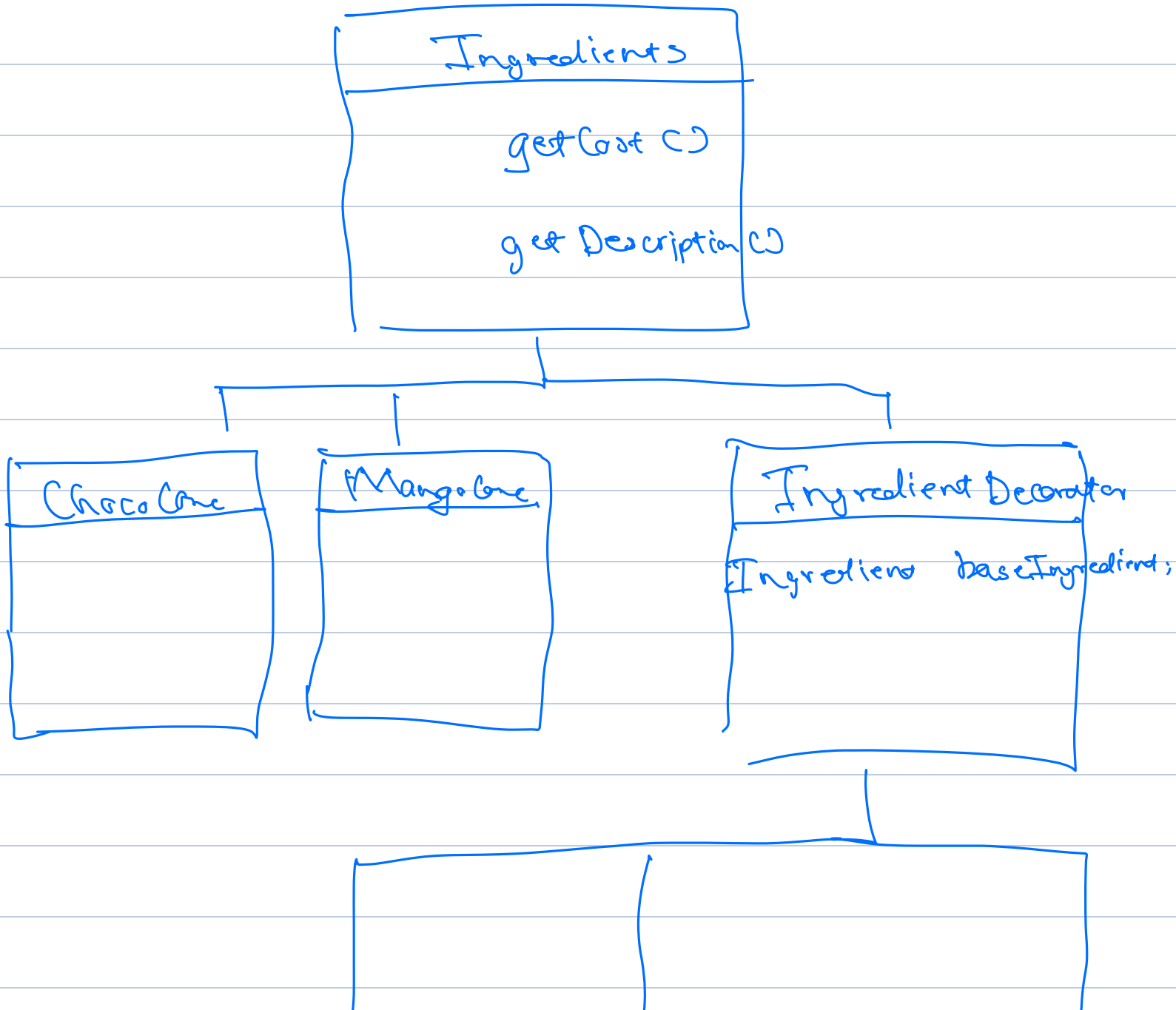
3

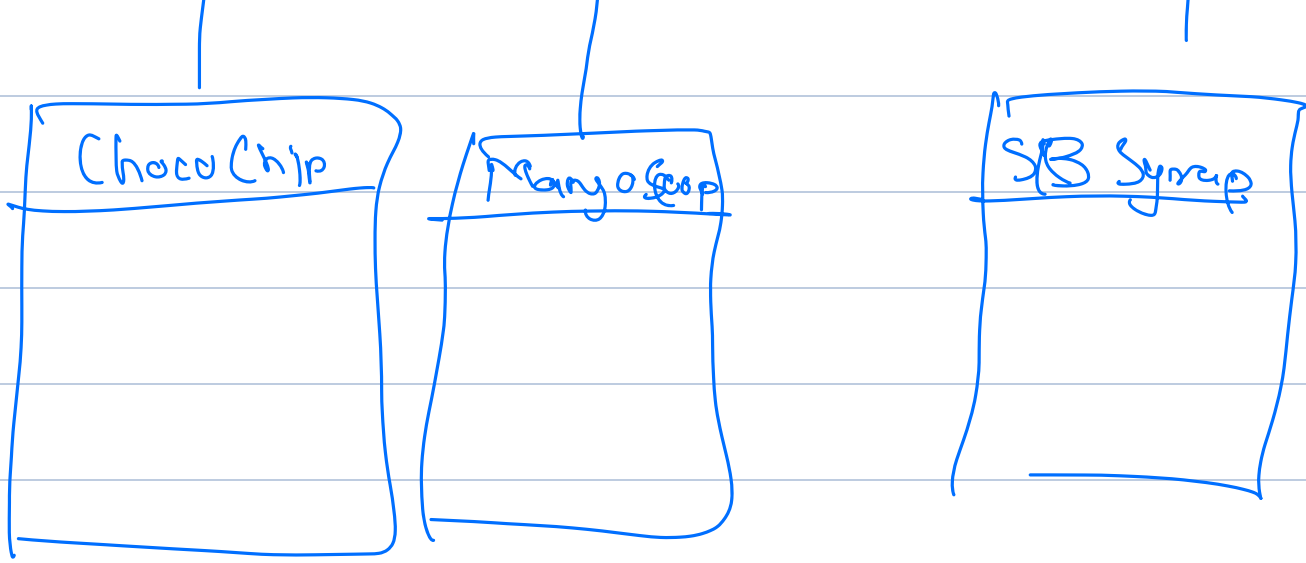




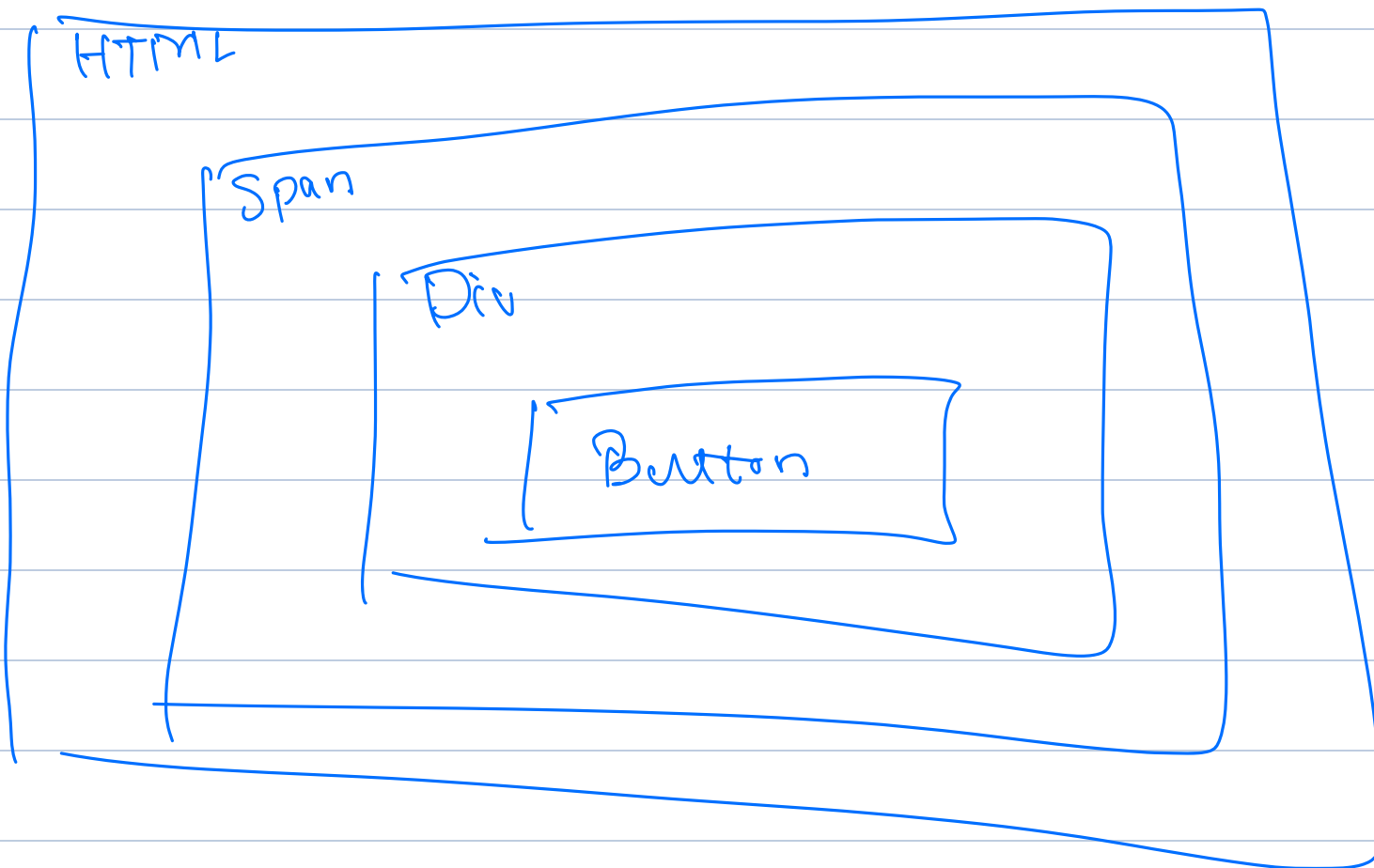
wall
painted wall
add furniture

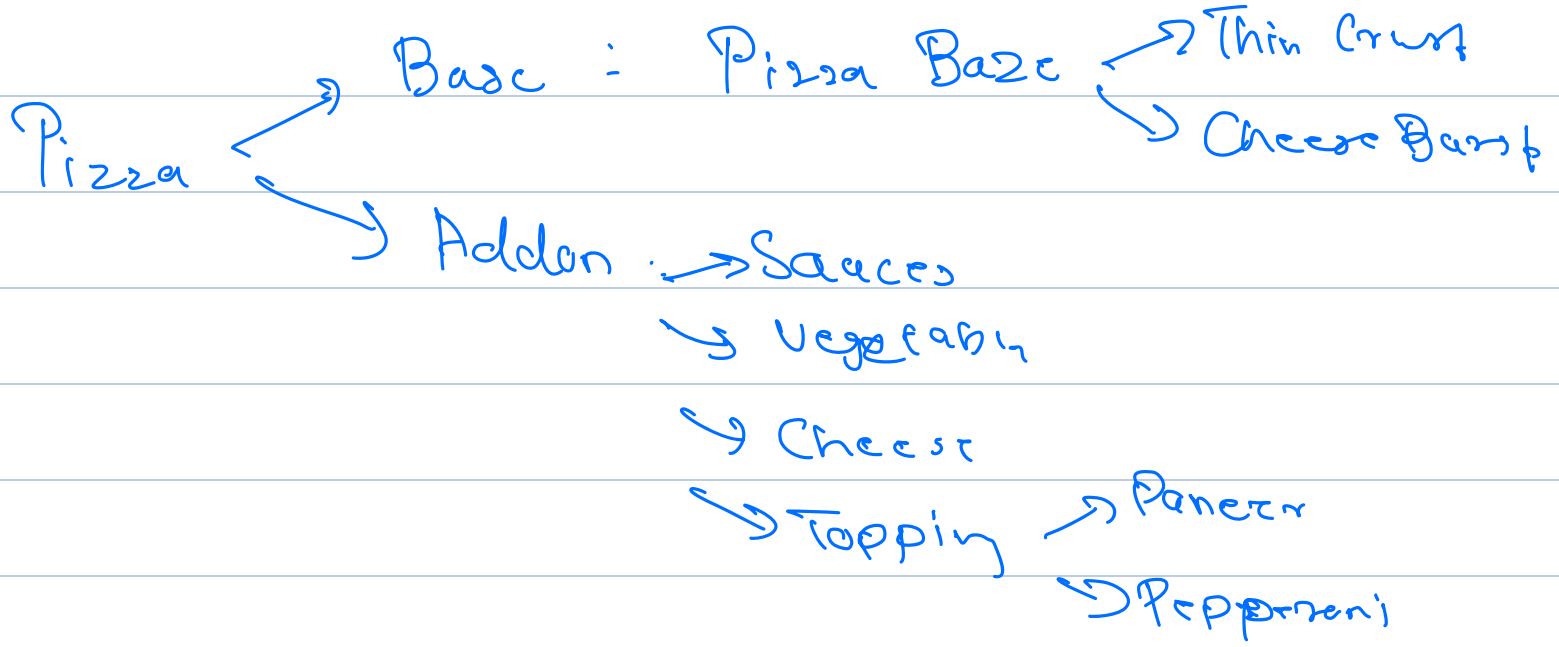
Decorator Design Pattern





Real World Example





Decorator design pattern: When we want to enhance an object & do it freely

Composition over Inheritance

1 small problem with current design:
20 Flavours of cones : 20 classes.

Bridge Design Pattern: solution

Mock Interview : LLD 1
(OOPS, Concurrency,
~~Adv. Java~~)

Flyweight Design Pattern



of Players : 100

Max # of guns in game : 2

Per gun # of bullets : 100

Per person # of bullets : 200

Min # of bullets in 1 game : 20,000

Max # of bullets in a game : 1,00,000

class Bullet {

4B ← BulletType type;

8B ← double range;

8B ← double damage;

8B ← double velocity;

24B ← Location currentLoc;

24B ← Location spawned

1024B ← byte[] image;

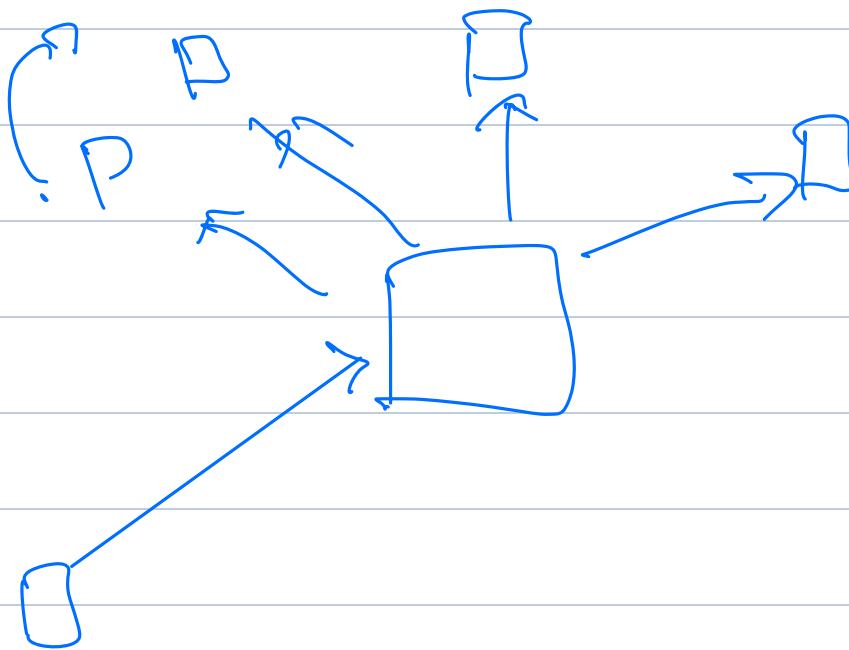
class Location {

double x, y,

z;

When a pubg game initializes:

- ① Backend Servers initialize all the objects (bullets, guns, buildings, vehicles)
- ② Entire game state is transferred to all the players locally.



Size of 1 Bullet = 1100 B

Size of 100,000 Bullets = $1100 \times 10^5 \text{ B}$
 $= 110 \times 10 \times 10^5 \text{ B}$
 $= 110 \times 10^6 \text{ B}$
 $= 110 \text{ MB}$

$1 \text{ MB} = 10^3 \text{ KB}$
 $= 10^3 \times 10^3 \text{ B}$

Every object has 2 types of properties:

- (1) intrinsic
- (2) extrinsic

Intrinsic

(Possible values for these attr are limited)

type, range, damage, image

Extrinsic

(Possible values are unlimited)

spawnedAt, currentLoc, velocity

↓ intrinsic
props

class Bullet {

8B ↙ double range;
8B ↙ double damage;
4B ↙ BulletType bulletType;
10B ↙ byte[] image;

1044B

}

↓ Bullet +
intrinsic

class FlyingBullet {

4B ↙ Bullet bullet;
24B ↙ Location spawned
At;
24B ↙ Location current
Loc;
8B ↙ double velocity;

60B

}

Types of bullets in a game : 4
Count per type : 25000

$$\begin{aligned}\text{Size occupied by bullet obj} &= 4 \times 1044 \text{ B} \\ &= 4176 \text{ B} \\ &= 4.1 \text{ kB}\end{aligned}$$

$$\begin{aligned}\text{Size occupied by flying bullet objects} &= 100k \times 60 \text{ B} \\ &= 10^5 \times 60 \text{ B} \\ &= 10^6 \times 6 \text{ B} \\ &= 6 \text{ MB}\end{aligned}$$

$$\begin{aligned}\text{Total space reqd} &= 4.1 \text{ kB} + 6 \text{ MB} \\ &= 6.004 \text{ MB} \\ &\approx 6 \text{ MB}\end{aligned}$$

Structural Design Patterns:

① Adapter:

to connect 2 incompatible things.

Whenever we want to use a 3rd party library.

② Facade:

Used to hide complex internal detail.

③ Decorator: whenever you want to enhance an object.

④ Flyweight: memory optimizations by dividing into intrinsic & extrinsic properties.

new Underline (

new Italic (

new Bold (

new Text ()

)

)

)

Goals for next class:

① Prototype & Registry assignment.

Assignments:

① Read up about decorator
& fly weight on refactoring.
guru

- ② Bridge design pattern
- ② Implement the icecream problem with bridge + decorator